AI class Final PDF report

Kay Chang (individual participant)

Prof. Wenpeng Yin

A-I 100

18 April 2026

Final

I decided to use my old code from my midterm project. So, I decided to reuse my midterm code here. Here are the 10 bug cases I came up with.

1. Wrong CSV file name

In my original file I have the code: EXCEL_PATH = "996ICU reworked data.xlsx"

In this case I can just change it to EXCEL_PATH = "996.xlsx”

The result would be the code unable to find the file

2. wrong column name in the code

In my original file I have the code: LABEL_COL = "label"

In this case I can change it to LABEL_COL = "labels"

The code will still run, but result will be distorted with drastically different result, because Label check what each column sees while labels look at multiple list or series of labels

3. Set the dropout rate to be too high

In my code there is a line: “layers.Dropout(0.3)”

My setting the code to: “layers.Dropout(0.9)”

This will cause the model to struggle to learn, accuracy stayed low, because too much information was randomly removed during training. Which make the data meaningless.

4. Remove stratify=labels in train/test split

```
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels,
    test_size=TEST_SIZE,
    random_state=RANDOM_SEED,
```

```
    stratify=labels if len(np.unique(labels)) > 1 else None
)
```

Remove stratify from this line of code will cause class distribution to become uneven and the final evaluation to be uneven.

5. Break the labeling system

There are codes in my file that is like return 0-2, Me changing the return number to something like 5 will break the system. The label will fall outside of expected range of 0-2. Which create inconsistency in the dataset and incorrect outputs.

6. wrong loss function

Change sparse_categorical_crossentropy to binary_crossentropy. So from muti-class classification to binary classification. This will cause one of category of data to be lost and the 2 other category's data to be corrupted with the 3rd category data.

7. missing or bad text data. (make the data in the excel file weird)

In the training data leave some data blank. Which will cause meaningless input, the code will still run, but the data quality break the entire point of the code.

8. wrong number output classes

Change the line of code: layers.Dense(num_classes, activation="softmax"), to layers.Dense(1, activation="sigmoid"). This will cause errors or meaningless training behavior.

9. very small vocabulary size

Change VOCAB_SIZE = 8000 to a very small number

This will cause the inability to analyze complex data, code will still run, but at the end of the day it will perform badly because of the small representation

10. Huge batch size (change batch size number up)

In my code the batch size is 16, if I change it to a huge number like 512, it will create problems like. The code might still work, if my computer allows it, but with a small training data huge batch size will create problems like crashing or less informative.

Overall, I learn that coding is a delicate matter, one wrong typo can crash the entire code. Many errors might seem harmless, because the code would still execute successfully, but it can destroy the quality of the model. Like for example when I changed LABEL_COL = "label" TO

LABEL_COL = "labels," the code still ran, but manual labels are not properly recognized, which could cause unpredicted training behavior. There are some easy errors, like when I changed TEXT_COL = "制度描述" to TEXT_COL = "wrongcolumnname" the error message literally said: “keyError: wrongcolumnname,” but the real challenge when it comes to training AI system have to be those “harmless” error, code still ran, and it is like a needle in a hay stack for me to find the error.